



МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

**Институт
информационных
технологий**

**Кафедра
информационных
технологий и
вычислительных систем**

Кочановский Кирилл Андреевич

Выпускная квалификационная работа
по направлению подготовки
09.04.04 «Программная инженерия»

Профиль: «Технологии разработки интеллектуальных систем и программных комплексов»

**ИССЛЕДОВАНИЕ МЕТОДОВ
АВТОМАТИЧЕСКОГО СОСТАВЛЕНИЯ
РАСПИСАНИЯ И РАЗРАБОТКА ПРОГРАММНОГО
СРЕДСТВА ПОДДЕРЖКИ ГЕНЕРАЦИИ
РАСПИСАНИЯ ДЛЯ МГТУ «СТАНКИН»**

Регистрационный номер № _____

Зав. кафедрой	_____	к.т.н., доц. Новоселова О. В.
Научный руководитель	_____	к.т.н., доц. Гаврилов А. Г.
Научный консультант	_____	доцент Тохунц Н. Б.
Обучающийся	_____	Кочановский К. А.

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

**Институт
информационных
технологий**

**Кафедра
информационных
технологий и
вычислительных систем**

«УТВЕРЖДАЮ»
Заведующий кафедрой ИТиВС
к.т.н., доц. Новоселова О. В.

«___» _____ 2026 г.

Задание

на выполнение выпускной квалификационной работы
по направлению подготовки
09.04.04 «Программная инженерия»

Профиль: «Технологии разработки интеллектуальных систем и программных комплексов»

Студент группы ИДМ-24-08
Кочановский К. А.

Научный руководитель
доцент, к.т.н., доц. Гаврилов А. Г.

**Тема: «Исследование методов автоматического составления
расписания и разработка программного средства поддержки
генерации расписания для МГТУ «СТАНКИН»**

Тема утверждена приказом «___». _____ 202_ г. № ____
Срок сдачи ВКР на кафедру «___». _____ 202_ г.

1. Описание задания на выполнение ВКР

1.1. Тип ВКР — исследовательская работа.

1.2. Цель исследования — повышение эффективности деятельности учебного заведения.

1.3. Объект исследования — программная поддержка процесса создания расписания учебных занятий.

1.4. Предмет исследования — архитектура и функционирование клиент-серверного веб-приложения для создания расписания учебных занятий с учётом личных предпочтений преподавательского состава.

1.5. Методы исследования — системный анализ, процессно-ориентированный подход и функциональное моделирование процессов, объектно-ориентированный анализ.

1.6. Задачи исследования:

1.6.1. Провести исследование различных методов автоматического составления расписаний учебных занятий. Сделать обоснованные выводы о наличии необходимости в разработке своего метода. Провести анализ правил составления расписания МГТУ «СТАНКИН».

1.6.2. Обосновать требования для клиент-серверного веб-приложения для создания расписания учебных занятий с учётом личных предпочтений преподавательского состава.

1.6.3. Разработать проект клиент-серверного веб-приложения для создания расписания учебных занятий с учётом личных предпочтений преподавательского состава.

1.6.4. Разработать клиент-серверное веб-приложение для создания расписания учебных занятий с учётом личных предпочтений преподавательского состава в соответствии с разработанным проектом.

2. Требования к выполнению ВКР

2.1. Соблюдение требований законодательной базы и стандартов

2.1.1. Образовательная программа высшего образования в магистратуре ФГБОУ ВО «МГТУ «СТАНКИН» по направлению подготовки 09.04.04 «Программная инженерия» для направленности (профиля) подготовки «Технологии разработки интеллектуальных систем и программных комплексов» (утв.

31.05.2021).

2.1.2. П 01-04/7/2024. Положение о государственной итоговой аттестации по образовательным программам высшего образования — программам бакалавриата, программам специалитета и программам магистратуры — утверждено приказом ректора от 03.04.2024 г. №700/1. — ФГБОУ ВО «МГТУ «СТАНКИН».

2.1.3. П 01-04/9/2024. Положение о выпускной квалификационной работе обучающихся по образовательным программам высшего образования — программам бакалавриата, программам специалитета, программам магистратуры — утверждено приказом ректора от 02.09.2024. №700/1. — ФГБОУ ВО «МГТУ «СТАНКИН».

2.2. Дополнительные требования

2.2.1. Оформление раздела «Список литературы» по национальному стандарту ГОСТ Р 7.0.100-2018 «Библиографическая запись. Библиографическое описание. Общие требования и правила составления»

2.2.2. Результаты исследования должны быть опубликованы в виде научных статей и тезисов докладов (не менее 4), а также доложены на научно-технических конференциях и семинарах.

3. Срок сдачи оформленной квалификационной работы на кафедру — вторая декада мая 2026 г.

Исполнитель

Кочановский К. А.

Научный руководитель

доцент каф. ИТиВС, к.т.н., доц.

Гаврилов А. Г.

Аннотация

выпускной квалификационной работы
студента группы ИДМ-24-08 ФГАОУ ВО «МГТУ «СТАНКИН»

Кочановского Кирилла Андреевича

по направлению подготовки

09.04.04«Программная инженерия»

на тему:

**«Исследование методов автоматического составления расписания и
разработка программного средства поддержки генерации расписания
для МГТУ «СТАНКИН»»**

В аннотации пишут обычно чему посвящена работа, освещают ее актуальность, дают характеристику задач.

ОГЛАВЛЕНИЕ

Перечень сокращений и специальных терминов	8
Введение	9
Глава 1. Анализ предметной области	10
1.1. Анализ методов автоматического создания расписания учебных занятий	10
1.1.1. Линейные и целочисленные программные методы составления расписания	11
1.1.2. Метаэвристики	11
1.1.3. Гиперэвристики	14
1.2. Выводы по первой главе	14
Глава 2. Разработка алгоритма генерации расписания	16
2.1. Минимальные требования к расписанию	16
2.2. Допустимое решение	18
2.2.1. Коллапс волновой функции	18
2.3. Оценка решения	18
2.4. Оптимизация решения	19
2.4.1. Имитация отжига	19
2.5. Вывод по второй главе	19
Глава 3. Разработка архитектуры сервиса для программной поддержки генерации расписания учебных заведений	20
3.1. Подход к созданию программного решения	20
3.2. Формирование архитектуры базы данных	20
3.2.1. ERD диаграмма базы данных	21
3.3. Вывод по третьей главе	22
Глава 4. Выбор средств реализации сервиса для программной поддержки генерации расписания учебных заведений	23
4.1. Обоснование выбора СУБД	23
4.1.1. Хранение данных в текстовых файлах	24

4.1.2. SQLite	24
4.1.3. Вывод	25
4.2. Выбор вычислительного движка	25
4.3. Выбор средств реализации сервера	26
4.4. Выбор средств реализации клиента	27
4.5. Вывод по четвёртой главе	28
Глава 5. Реализация сервиса для программной поддержки генерации расписания учебных заведений	29
5.1. Заполнение базы данных	29
5.1.1. Процесс популяции базы данных	29
5.2. Сервер	30
5.2.1. Реализация WFC на языке Polars	31
5.3. Клиент	36
5.4. Вывод по пятой главе	37
Глава 6. Руководство пользователя	38
Глава 7. Экономическое обоснование	39
Заключение	40
Список литературы	41
Приложение А. Первое приложение	44
Приложение Б. Пример листинга в приложении.	45

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И СПЕЦИАЛЬНЫХ ТЕРМИНОВ

Сокращения

UI (User Interface) — пользовательский интерфейс

WASM (WebAssembly) — бинарный формат, запускаемый в браузере

БВО (Базовое высшее образование) — соответствует бакалавриату и специалитету

СВО (Специализированное высшее образование) — уровень, на котором реализуются программы магистратуры, ординатуры и ассистентуры-стажировки

СУБД (Система управления базами данных)

ERD (Entity-Relationship Diagram) — модель данных, позволяющая описывать концептуальные схемы предметной области

ЭОС (Электронная образовательная среда)

Термины

Fitness function функция приспособленности — критерий качества решения в оптимизации или ML

Бекэнд часть программного обеспечения или веб-приложения, отвечающая за серверную логику, обработку данных и взаимодействие с базами данных

Фреймворк структурированный набор инструментов и библиотек, облегчающий разработку программного обеспечения

Хостинг услуга или платформа для размещения веб-сайтов и приложений на сервере, обеспечивающая их доступность в сети Интернет

Парсинг — автоматизированный сбор и структурирование информации

ВВЕДЕНИЕ

Формирование расписания учебных занятий является одной из основных и наиболее сложных задач автоматизации управления учебным процессом вуза. Каждый семестр специальный отдел в составе Учебно-методического управления МГТУ «СТАНКИН» [1] составляет расписание на следующие полгода, выкладывает его на сайт университета и отправляет преподавателям и старостам групп. На сегодняшний день процесс создания расписания учебных занятий производится вручную, что является медленным и ресурсозатратным действием. В моей выпускной квалификационной работе я рассмотрю разные методы автоматического создания расписания, а также реализую сервис, выполняющий следующие функции:

1. Генерация эффективного расписания с учётом учебных планов, нормативов, списков преподавателей, групп и т. д.
2. Возможность выбора разгрузочных дней преподавателями.
3. «Компоновка вуза» для оптимизации временных затрат на передвижение между парами.
4. Вывод расписания в различных форматах, включая веб-интерфейс для возможности просмотра расписания преподавателями и студентами.

Так как качество расписания занятий определяет эффективность всего учебного процесса и работы вуза, немало внимания будет уделено оценке расписаний и их последующему улучшению.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. АНАЛИЗ МЕТОДОВ АВТОМАТИЧЕСКОГО СОЗДАНИЯ РАСПИСАНИЯ УЧЕБНЫХ ЗАНЯТИЙ

Задача создания расписания сводится к формированию и оптимизации процесса обслуживания конечного множества требований (заявок) на осуществление действий в системе. Эта задача является задачей распределения трёх ресурсов – преподавательского состава, студенческого состава и аудиторий. В детали создания расписания я углублюсь в следующий главе, задача сейчас – определить какие методы для этого существуют и придумать свой подход.

Если посмотреть на количество статей (см. рис. 1.1), публикуемых международным сообществом на эту тему за последний десяток лет [2], то можно сделать вывод, что область образовательной логистики всё ещё является актуальной темой исследований и разработки.

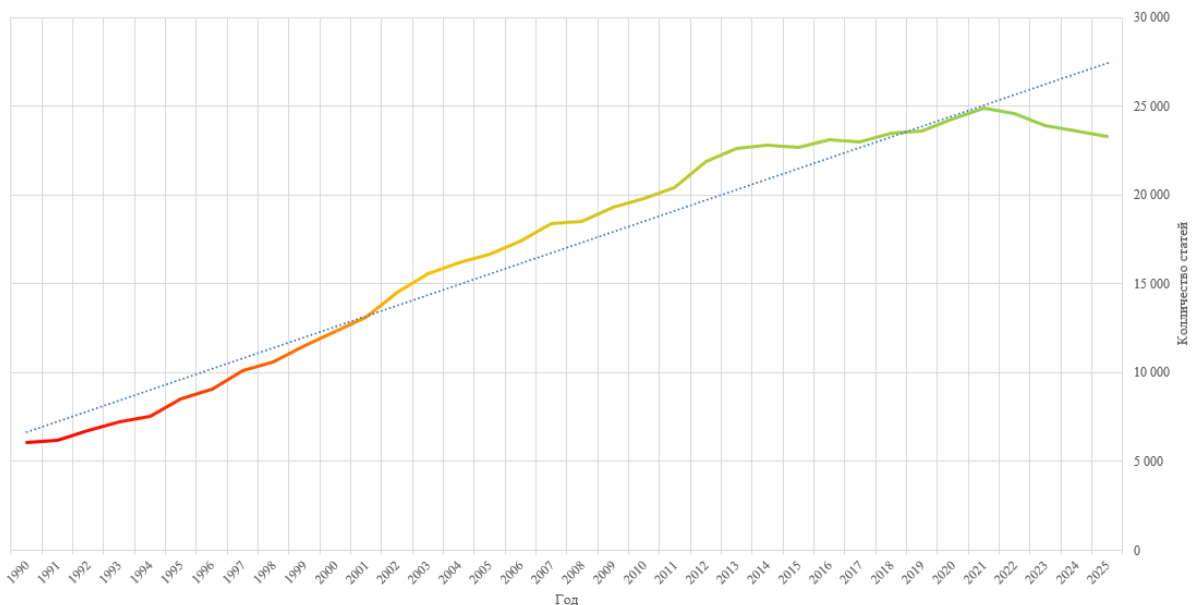


Рис. 1.1. Количество статей по годам на тему расписаний в высших учебных заведениях

Для того, чтобы разобраться на каком этапе сейчас эта область, стоит

проанализировать различные подходы и выявить основные тенденции и идеи.

1.1.1. Линейные и целочисленные программные методы составления расписания

Одними из первых, кто предоставил стратегии раскраски графов для решения проблемы составления расписания были Уэлш и Пауэлл (1967) [3]. Они заложили основу для более сложных исследований графовых эвристик в составлении расписания [4]. Эвристики составления расписания на основе раскраски графов - это полезный метод, на котором строится их оценка и улучшение.

Линейные и целочисленные программные методы - это математические алгоритмы, которые присваивают целочисленные значения переменным. Вариации этого метода были и до сих пор широко используются для решения комбинаторных задач оптимизации. Эти методы включают в себя программирование с ограничениями и методы удовлетворения ограничений.

Однако такие методы, как правило, требуют больших вычислительных ресурсов из-за экспоненциального количества переменных.

1.1.2. Метаэвристики

Метаэвристика – это метод оптимизации, многократно использующий простые правила или эвристики для достижения оптимального или субоптимального решения. С 1995 по 2010 годы методологические подходы к решению проблем составления расписания обычно классифицировались на две категории метаэвристик: популяционные и одиночные [5]. Если классифицировать основные подходы и алгоритмы, то можно построить следующий график (см. рис. 1.2).

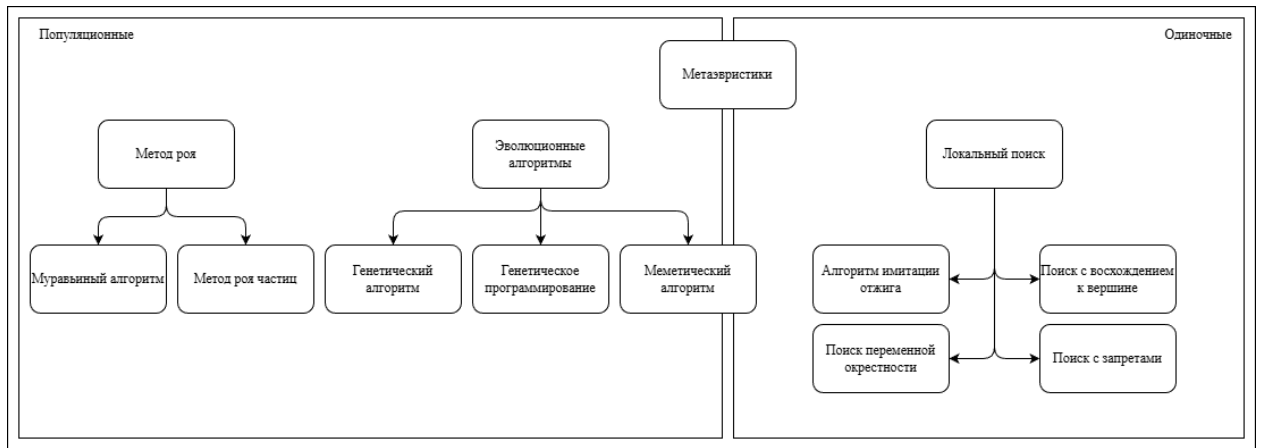


Рис. 1.2. Классификация алгоритмов решения задачи

Метаэвристический подход позволяет быстро получить приемлемое решение популяционным алгоритмом, а в последствии улучшать и удовлетворять дополнительные ограничения итеративным образом используя одиночные методы.

Естественно нет возможности окунуться с деталями в каждый подход, но стоит хотя бы поверхностно изучить и оценить разные метаэвристические методы.

1.1.2.1. Локальные алгоритмы

Поиск с запретами. Поиск с запретами (Tabu search) относится к методологиям локального поиска и основан на поиске градиентного спуска. Существует несколько актуальных работ, в которых проведено ценное исследование методов поиска с запретами [6, 7]. Исследования основаны на диверсификации соседей, при котором поиск расширяется для нахождения большего количества локальных оптимумов. Это приводит к интенсификации шагов и более быстрого нахождения решения.

Однако поиск с запретами имеет тенденцию застрять в подоптимальных областях или плато. Подоптимальные области - это регионы пространства поиска, где текущее решение локально оптимально, но не обязательно глобально. Плато представляют собой плоские области функции, где градиент близок к нулю, что затрудняет продвижение поиска.

Алгоритм имитации отжига. Алгоритм имитации отжига (Simulated annealing – SA) – это ещё один метод локального поиска. Имитация отжига вдохновлена физическим процессом отжига металла, при котором нагреваемый металл постепенно остывает, обеспечивая более устойчивую кристаллическую структуру. В контексте оптимизации, этот метод направлен на поиск более широкой области пространства поиска, в которой принимаются худшие шаги и допускается более широкий поиск оптимального решения. Существует множество вариаций SA [5, 8], он часто комбинируется с алгоритмом восхождения к вершине [9], или программированием с ограничениями [10].

Имитация отжига может стать важной частью моего будущего алгоритма, на процессе оптимизации расписания и увеличения его равномерности.

1.1.2.2. Эволюционные алгоритмы

Генетические алгоритмы. Среди эволюционных алгоритмов одними из самых изученных и популярных являются генетические алгоритмы (Genetic algorithms) [11]. Эти алгоритмы моделируют процесс естественного отбора, таким образом находя более совершенную форму «жизни».

Меметические алгоритмы. Как дополнение к генетическим алгоритмам рассматриваются меметические алгоритмы [12]. Меметические алгоритмы (Memetic algorithms) представляют собой эволюционный метод оптимизации, который объединяет генетические алгоритмы с понятием «мема» из теории эволюции Ричарда Докинза. В данном контексте мем – это небольшая единица знания или идеи, которая может передаваться от одного индивида к другому.

Муравьиный алгоритм. Другой метод, основанный на популяции и более подробно исследованный в последнее десятилетие – это группа муравьиных алгоритмов (Ant algorithms) [13]. Эти алгоритмы отслеживают информацию, собранную в процессе поиска, и затем используют эту информацию для создания новых решений на следующих этапах.

1.1.3. Гиперэвристики

Как для подходов, основанных на одиночных решениях, так и для подходов, основанных на популяции, характерны свои недостатки. В последнее время большинство исследователей в области составления расписаний сосредотачивались на локальных или одиночных решениях, а не на алгоритмах, основанных на популяции [14].

Развитием, вытекающим из стандартных локальных и населённых решений проблем составления расписаний, является применение гиперэвристик. Гиперэвристики представляют собой метод поиска, в котором несколько эвристик объединяются и адаптируются. Различие между метаэвристиками и гиперэвристиками заключается в том, что гиперэвристики стремятся найти общепринятую методологию, а не решать конкретный экземпляр проблемы.

1.2. ВЫВОДЫ ПО ПЕРВОЙ ГЛАВЕ

Был проведён анализ различных подходов автоматического создания расписания как у отечественных высших учебных заведений, так и в международных. По результатам анализа был сделан вывод, что лучшим подходом будет метаэвристика, то есть объединение сразу нескольких алгоритмов для достижения цели. Как показывает практика последних лет, лучше всего подходят одиночные алгоритмы работающие в локальной области поиска.

После проведенного анализа было принято решение о создании собственного программного средства поддержки генерации расписания

для МГТУ «СТАНКИН». Одним из кандидатов на метод генерации расписания является алгоритм коллапса волновой функции (Редукция фон Неймана) для получения первичного решения, а дальнейшая оптимизация для повышения эффективности расписания будет проводиться алгоритмом имитации отжига.

ГЛАВА 2. РАЗРАБОТКА АЛГОРИТМА ГЕНЕРАЦИИ РАСПИСАНИЯ

Для автоматического создания расписания будет использоваться метаэвристический подход, комбинирующий в себе сразу несколько алгоритмов.

Сперва будет получено первичное приемлемое решение – то есть решение, удовлетворяющее минимальным требованиям. Допустимым решением для поставленной задачи будет являться расписание, содержащее в себе все дисциплины согласно учебному плану и не нарушающее никакие нормативы.

Далее итеративным методом расписание будет оцениваться и улучшаться путём перестановки различных учебных занятий местами.

2.1. МИНИМАЛЬНЫЕ ТРЕБОВАНИЯ К РАСПИСАНИЮ

В этой главе будут рассмотрены строгие требования к расписанию работы учебного заведения. Расписание на семестр должно строго соответствовать учебному плану и количеству часов лекций, семинаров и лабораторных занятий. При этом нельзя просто «запихнуть» по 10 пар каждый день, существуют нормативы времени по нагрузке как на преподавателей, так и на студентов. Также нельзя забывать и о рабочих часах самого учебного заведения, пары должны начинаться в строго отведённое под них время с 8:30 утра, по 22:50, без учёта выходных дней и праздников. Список нерабочих праздничных дней задаётся «Трудовым кодексом Российской Федерации» от 30.12.2001 № 197-ФЗ (ред. от 14.02.2024) [15]. Упрощённая диаграмма документов и таблиц управления учебным процессом приведена на рисунке ниже (см. рис. 2.1).

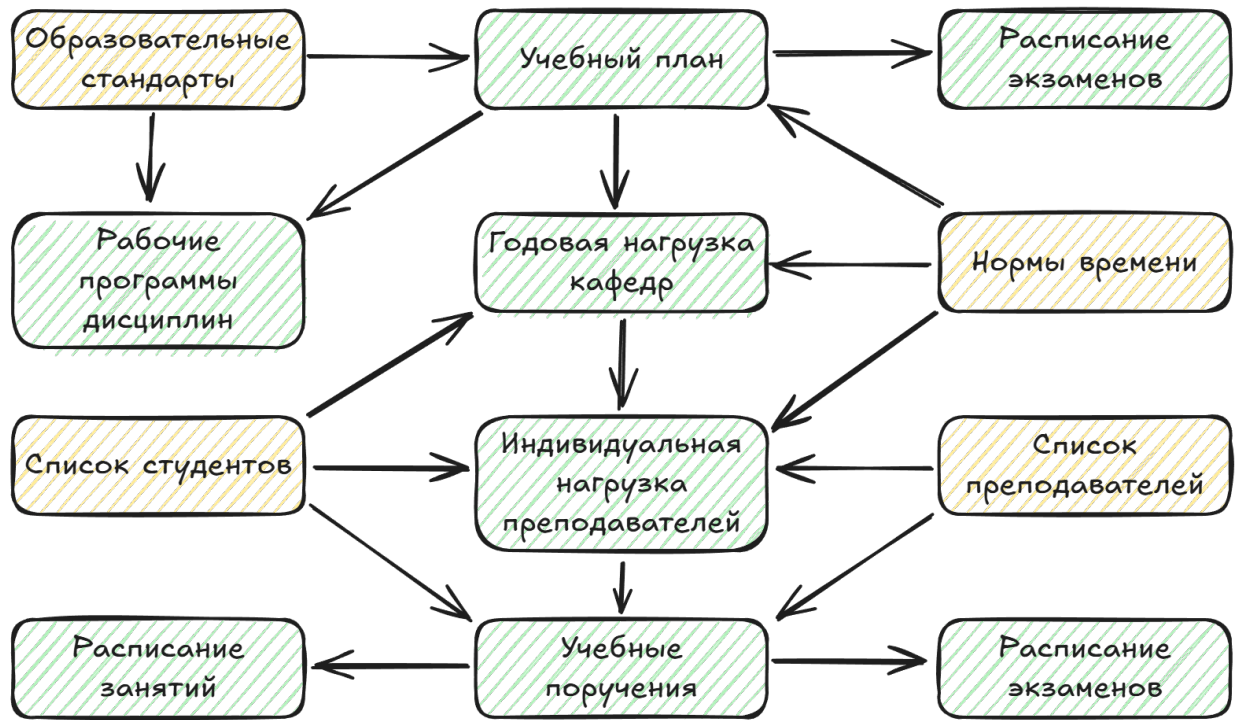


Рис. 2.1. Диаграмма документов и таблиц управления учебным процессом

Список студентов будет делиться на направления, согласно учебным планам, а также на группы и подгруппы для проведения семинаров и лабораторных занятий соответственно. Следует подметить, что деление на подгруппы является рудиментарным процессом, изначальный смысл которого заключался в невозможности вместить достаточное количество студентов в классы с компьютерами.

Не менее важным является подбор аудиторий для проведения учебных занятий. Главным параметром каждой аудитории является вместимость – нельзя назначать учебное занятие в аудиторию, которая физически не может вместить весь контингент студентов и преподавателей на данное учебное занятие. Дополнительным, не менее важным параметром является требуемое оборудование для проведения учебного занятия: мультимедийное и лабораторное оборудование.

Схематичное дробление контингента студентов и здания вуза предоставлено на рисунке ниже (см. рис. 2.2).

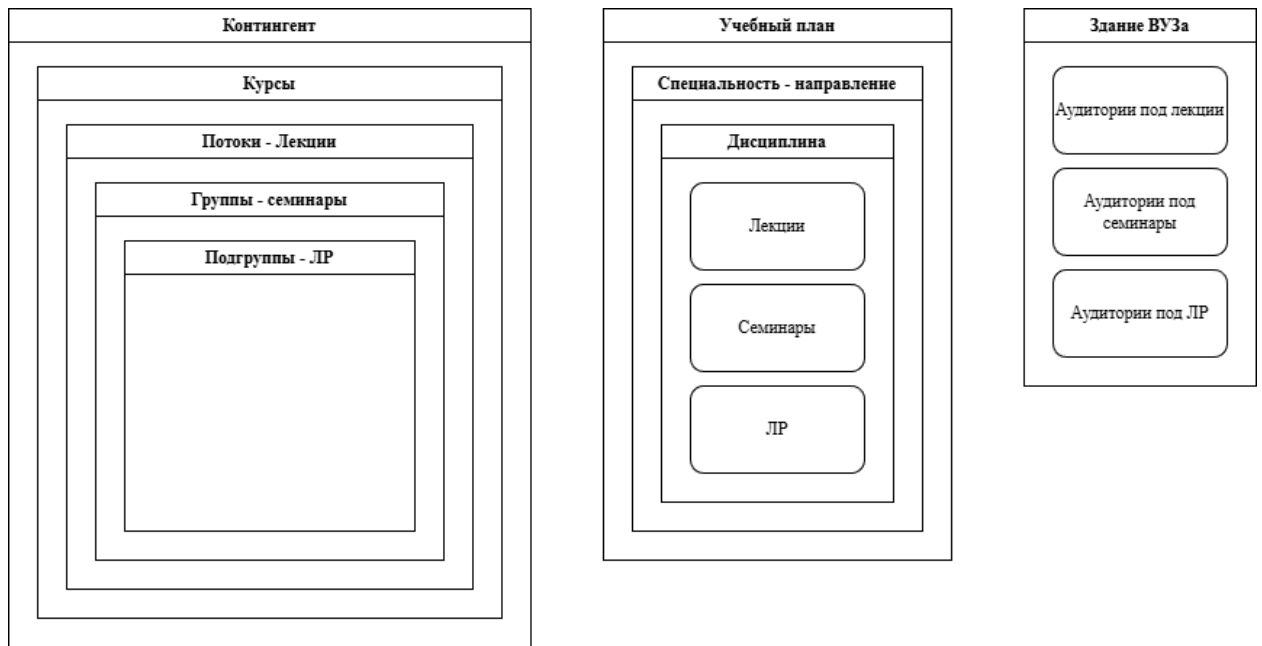


Рис. 2.2. Схематичное дробление контингента и здания вуза

2.2. ДОПУСТИМОЕ РЕШЕНИЕ

2.2.1. Коллапс волновой функции

Коллапс волновой функции (Wave Function Collapse, WFC) [16], он же редукция фон Неймана, является алгоритмом, имитирующим мгновенное изменение описания квантового состояния (волновой функции) объекта при его измерении. Это очень полезный алгоритм и вне физических симуляций. Например, для генерации расписания, так как при выборе конкретной пары, происходит как раз таки процесс коллапса этой пары из суперпозиции всех возможных пар на данную временную ячейку, с учётом всех ограничений, в конкретную дисциплину и тип занятия.

2.3. ОЦЕНКА РЕШЕНИЯ

После получения первичного решения, которое удовлетворяет всем обязательным требованиям требуется провести итеративный процесс улучшения этого расписания. Для того, чтобы что-либо улучшать, нужно уметь давать оценку полученным результатам.

Для оценки будет рассчитываться множество различных параметров. Одним из самых главных таких параметров является непрерывность расписания – в идеальном расписании будущего не должно быть «окон», на которых студенты имеют свойство расходиться по домам.

Ещё один важный параметр – это равномерность расписания. Не должно быть расписания, где в понедельник пять пар, а во вторник только одна. Согласно исследованию, проведённому Клеванским Николаем Николаевичем [17] студенты предпочитают, когда одни и те же пары проводятся в одно и то же время в разные дни недели. Соответственно существует три оценки равномерности:

- равномерность загруженности;
- равномерность начала занятий;
- равномерность идентичности пар по дням / неделям.

Дополнительным параметром может являться «компоновка» ВУЗа. Для экономии временных и финансовых ресурсов можно попробовать уместить весь учебный процесс в меньший набор аудиторий. Также можно пробовать уменьшать среднюю дистанцию, требуемую для перехода из одной аудитории в другую между учебными занятиями. В случае проведения лабораторных занятий дистанционно можно не делить на подгруппы и т. д.

2.4. ОПТИМИЗАЦИЯ РЕШЕНИЯ

2.4.1. Имитация отжига

2.5. ВЫВОД ПО ВТОРОЙ ГЛАВЕ

ГЛАВА 3. РАЗРАБОТКА АРХИТЕКТУРЫ СЕРВИСА ДЛЯ ПРОГРАММНОЙ ПОДДЕРЖКИ ГЕНЕРАЦИИ РАСПИСАНИЯ УЧЕБНЫХ ЗАВЕДЕНИЙ

3.1. ПОДХОД К СОЗДАНИЮ ПРОГРАММНОГО РЕШЕНИЯ

Для разработки будущего сервиса была выбрана трёхзвенная архитектура (three-tier architecture):

1. База данных (data layer) – слой для хранения исходных данных, требуемых для генерации расписания, а также для хранения уже готовых расписаний.
2. Сервер (application layer) – на этом слое реализуется вся бизнес-логика и REST API эндпоинты для работы по протоколу HTTP(S).
3. Клиент (presentation layer) – финальный слой, необходимый для отображения интерфейса сервиса, для удобства взаимодействия с ним через браузер.

Подходом к написанию кода было выбрано объектно-ориентированное программирование (ООП). Таким образом все сущности, такие как расписание, учебная аудитория, преподаватель – становятся объектами, обладающими некими параметрами и методами. Это сильно упрощает логику и инкапсулирует сложную логику внутри самих объектов.

3.2. ФОРМИРОВАНИЕ АРХИТЕКТУРЫ БАЗЫ ДАННЫХ

Прежде чем создавать таблицы базы данных и нормализировать их требуется определиться с данными, которые нужно хранить.

Есть четыре основные сущности:

- группа;
- преподаватель;
- аудитория;
- учебное занятие.

Каждая из сущностей должна содержать следующие данные:

1. Группа (group):

- имя группы;
- направления;
- институт;
- количество студентов.

2. Преподаватель (lecturer):

- институт;
- должность;
- ФИО.

3. Аудитория (classroom):

- номер аудитории;
- вместимость аудитории;
- оборудование.

4. Учебное занятие (subject):

- преподаватель;
- количество лекций / семинаров / лабораторных занятий;
- требуемое оборудование.

3.2.1. ERD диаграмма базы данных

Определив сущности и связи между ними можно построить ERD диаграмму архитектуры базы данных и привести её к третьей нормальной форме (см. рис. 3.1).

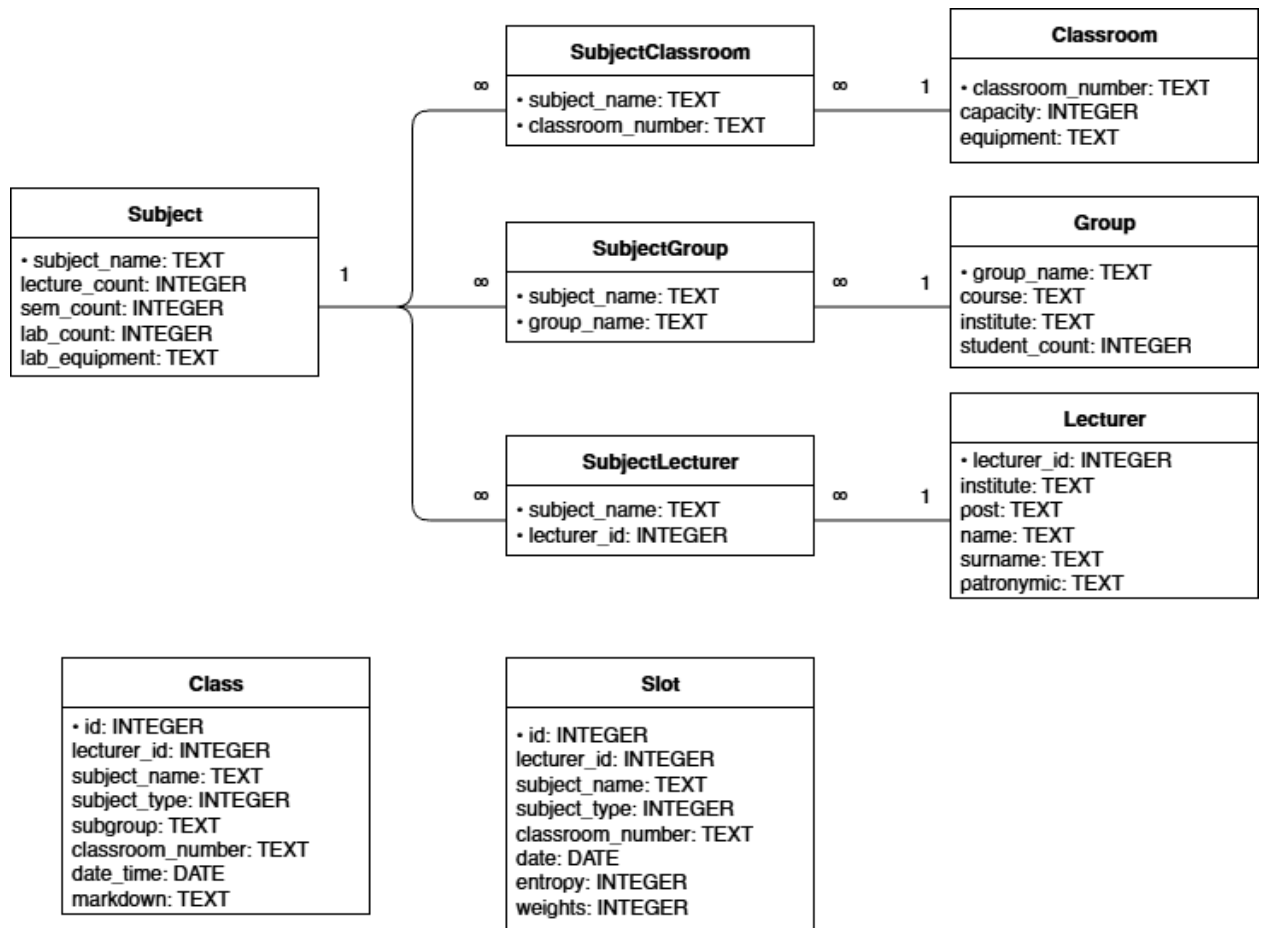


Рис. 3.1. ERD диаграмма базы данных

3.3. ВЫВОД ПО ТРЕТЬЕЙ ГЛАВЕ

Была сформирована архитектура сервиса для генерации расписания учебных занятий. Для системы была выбрана трёхзвенная архитектура, обеспечивающая разделение ответственности между хранилищем данных, серверной логикой и пользовательским интерфейсом. Также была продумана архитектура базы данных и построена ERD диаграмма зависимостей между сущностями.

ГЛАВА 4. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ СЕРВИСА ДЛЯ ПРОГРАММНОЙ ПОДДЕРЖКИ ГЕНЕРАЦИИ РАСПИСАНИЯ УЧЕБНЫХ ЗАВЕДЕНИЙ

4.1. ОБОСНОВАНИЕ ВЫБОРА СУБД

При выборе СУБД требуется учесть следующие факторы:

1. СУБД должна быть реляционной. Между таблицами студентов, преподавателей, аудиторий и предметов существует большое количество связей типа «многие ко многим», без обеспечения ссылочной целостности (foreign key integrity) работа с такими данными была бы существенно затруднена.
2. Требуется высокая скорость чтения данных. Как алгоритм коллапса волновой функции, так и генетический алгоритм требуют выполнения большого количества запросов, связанных с выборкой и фильтрацией данных, а также объединение таблиц. При этом характерной особенностью задачи составления учебного расписания является относительно небольшой объём данных: как правило, он составляет считанные мегабайты и полностью помещается в оперативной памяти (RAM).
3. Размер данных: для генерации расписания не требуется огромный объём данных. Учебные планы, списки групп, списки преподавателей и т. д. занимают относительно небольшое место на диске.
4. Тип данных: данные для генерации расписаний являются текстовыми данными и легко могут храниться в любой СУБД или даже просто в текстовом файле, как JSON или CSV.
5. Простота использования: главной задачей ВКР является сам алгоритм генерации расписания, база данных является лишь второстепенной задачей, которая не должна отнимать много времени на развёртывание и заполнение.

Учитывая вышеперечисленные факторы, можно прийти к выводу, что

для поставленной задачи требуется небольшая, легковесная БД с высокой скоростью чтения.

4.1.1. Хранение данных в текстовых файлах

Хранения данных в текстовых файлах, таких как JSON, TXT или CSV отлично подходит для поставленной задачи. Из плюсов можно выделить простоту использования, подходит для хранения сложных, вложенных структур данных, легко читается человеком и легко вносить изменения в структуру данных.

Однако есть и минусы. При каждом чтении будет производиться операция парсинга текстовых данных, что будет делать операции чтения, изменения или добавления сравнительно медленными. Более того, такие, классические для СУБД вещи как поиск, индексация, транзакции отсутствуют, а сами данные будут храниться не в бинарном файле, а в текстовом, что будет неоправданно раздувать объём хранимых данных.

Текстовый формат данных отлично подходит для транспортировки небольших объёмов данных, таких как уже сгенерированное расписание, на клиент для дальнейшего отображения расписания учебных занятий. Однако текстовые файлы не идеально подходят для хранения и работы с данными, используемыми в процессе генерации расписания.

4.1.2. SQLite

SQLite – это самодостаточная и встроенная СУБД [18]. Для развёртывания которой не требуется сервер или сложная система управления. Все данные хранятся в бинарном файле прямо в проекте с расширением “.sqlite” или “.db”.

Благодаря минимальным накладным расходам и высокой скорости выполнения операций чтения SQLite хорошо подходит для задач, в которых данные целиком помещаются в оперативной памяти.

Из минусов часто выделяют ограничение по объёму данных до

140Тб, однако это не будет проблемой для хранения данных для генерации расписаний учебных занятий. Также SQLite не подходит для сложных многопользовательских сценариев – отсутствует полноценная поддержка конкурентности.

4.1.3. Вывод

При небольшом анализе вариантов СУБД для хранения данных проекта выбор пал на SQLite. Это легковесная СУБД, обладающая всеми преимуществами классических СУБД как индексация и скорость обработки запросов, но и в то же время обладающая всеми преимуществами хранения данных в текстовых файлах – простота и гибкость. Более того, у автора данной выпускной квалификационной работы уже был опыт работы с SQLite, а соответственно не будет проблем с использованием, связанных с низким уровнем компетенции в инструменте.

4.2. ВЫБОР ВЫЧИСЛИТЕЛЬНОГО ДВИЖКА

Предлагаемый алгоритм решения задачи является ресурсоёмким, поэтому оптимизация производительности системы является одним из ключевых требований. По состоянию на 2026 год одним из наиболее эффективных инструментов для обработки табличных данных в рамках одной машины является библиотека Polars [19].

Изначально Polars был разработан Ричи Винком (Ritchie Vink) как экспериментальный проект во время пандемии COVID-19. Первоначальной целью было создание быстрого парсера CSV-файлов на компилируемом языке Rust, однако со временем проект вырос в полноценный высокопроизводительный движок обработки данных. Polars реализует вычисления с использованием ленивых выражений и оптимизаций на уровне плана выполнения, что позволяет существенно ускорить обработку данных.

Также Polars предоставляет интерфейс для Python, синтаксис которого

во многом схож с Pandas. Использование декларативного подхода позволяет абстрагироваться от низкоуровневых деталей реализации таких операций, как агрегация, фильтрация и соединение таблиц (JOIN), что в свою очередь упрощает разработку и повышает читаемость кода.

4.3. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ СЕРВЕРА

Поскольку основная вычислительная нагрузка в системе возлагается на движок обработки данных Polars, выбор языка программирования и серверного фреймворка в меньшей степени определяется требованиями к производительности. В данном случае приоритетными становятся такие критерии, как скорость разработки и простота сопровождения.

В качестве языка программирования был выбран Python благодаря его простому синтаксису и широкому набору библиотек. Кроме того, Python имеет нативную интеграцию с Polars, что упрощает организацию обработки данных и снижает накладные расходы на взаимодействие между компонентами системы. Для реализации серверной части был выбран фреймворк FastAPI [20]. Он предоставляет удобные средства для создания REST API, автоматическую валидацию входных данных на основе аннотаций типов, а также встроенную генерацию документации (Swagger).

Дополнительно для управления зависимостями и окружением был использован инструмент UV. Он представляет собой современную альтернативу традиционным решениям, таким как pip и virtualenv, обеспечивая значительно более высокую скорость установки пакетов и воспроизводимость окружения. Использование UV позволяет упростить процесс развёртывания проекта и снизить время настройки среды разработки.

Таким образом, использование Python в сочетании с FastAPI позволяет быстро разрабатывать и масштабировать серверную часть приложения, обеспечивая при этом достаточный уровень производительности и удобство взаимодействия с клиентской частью.

4.4. ВЫБОР СРЕДСТВ РЕАЛИЗАЦИИ КЛИЕНТА

При выборе стека для разработки клиентской части основными критериями являлись современность используемых технологий, удобство разработки и поддержка типобезопасности. В результате был выбран стек, включающий Nuxt 4, Vue 3.5, TypeScript и пакетный менеджер `pnpm`.

Фреймворк Nuxt [21] предоставляет высокоуровневую архитектуру для построения приложений на основе Vue. Он включает в себя готовые решения для маршрутизации, серверного рендеринга и управления состоянием, что позволяет сосредоточиться на реализации бизнес-логики, а не на настройке инфраструктуры проекта. Новые версии Nuxt ориентированы на упрощение разработки за счёт автоматической конфигурации и минимизации шаблонного кода.

Использование Vue версии 3.5 обеспечивает доступ к современным возможностям фреймворка, таким как Composition API, улучшенная реактивность и более гибкая организация кода. Это способствует повышению читаемости и переиспользуемости компонентов.

Применение TypeScript добавляет статическую типизацию, что снижает количество ошибок на этапе разработки и упрощает сопровождение проекта. Типизация особенно полезна при взаимодействии с API, где важно строгое соответствие структуры данных.

В качестве пакетного менеджера был выбран `pnpm`, который отличается высокой скоростью работы и эффективным использованием дискового пространства за счёт механизма кеширования зависимостей. Это ускоряет установку пакетов и делает процесс разработки более предсказуемым. Таким образом, выбранный стек сочетает в себе современные подходы к разработке веб-приложений и удобство использования, позволяя минимизировать технические издержки и сосредоточиться на реализации функциональности пользовательского интерфейса.

4.5. ВЫВОД ПО ЧЕТВЁРТОЙ ГЛАВЕ

Был сформирован стек технологий, обеспечивающий эффективную реализацию системы генерации расписания учебного заведения. Также был произведён сравнительный анализ подходящих для проекта СУБД и сделан обоснованный выбор на SQLite.

Выбранные инструменты позволяют решать ключевые задачи проекта: обработку и трансформацию данных, реализацию алгоритмов оптимизации, организацию клиент-серверного взаимодействия и визуализацию результатов. Использование современных решений обеспечивает достаточную производительность системы при работе с большим количеством входных данных.

ГЛАВА 5. РЕАЛИЗАЦИЯ СЕРВИСА ДЛЯ ПРОГРАММНОЙ ПОДДЕРЖКИ ГЕНЕРАЦИИ РАСПИСАНИЯ УЧЕБНЫХ ЗАВЕДЕНИЙ

5.1. ЗАПОЛНЕНИЕ БАЗЫ ДАННЫХ

В идеальном сценарии для заполнения базы данных можно использовать API систем университета, хранящих такие данные как списки групп, преподавателей и учебные планы. Однако на момент написания данной выпускной квалификационной работы такая возможность отсутствует.

Альтернативный вариант – производить парсинг PDF файлов учебных планов и календарных учебных графиков, которые находятся в открытом доступе на сайте университета [22, 23].

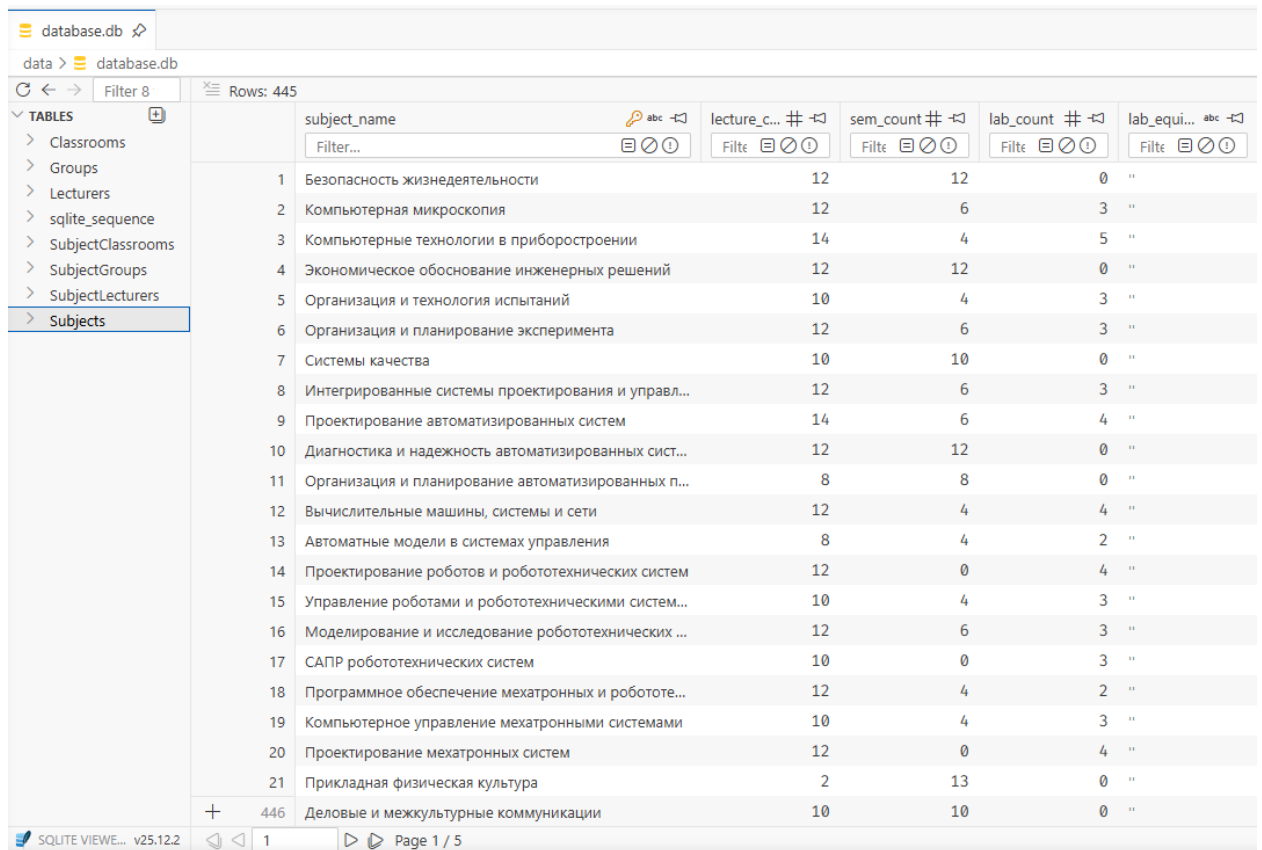
Однако существует третий вариант: переиспользовать написанный автором данной выпускной квалификационной работы в бакалавриате парсер [24] PDF файлов расписаний учебных занятий. Хотя это и не будет финальным решением, готовым для запуска системы в работу на пользу учебного заведения – это будет путём наименьшего сопротивления.

Этот подход также решит две другие проблемы, не имеющие элегантного решения. Тогда, как списки групп можно получить из соответствующего форума в ЭОС ФГБОУ ВО «МГТУ «СТАНКИН» [25], актуальные списки преподавательского состава или таблицы со всеми учебными аудиториями и оборудованием найти не получится. В случае получения данных сразу из готового расписания за прошлый семестр можно определить и какие предметы ведут какие преподаватели, и какие предметы проводятся в каких аудиториях.

5.1.1. Процесс популяции базы данных

Скачать все расписания на текущий семестр можно в соответствующем форуме в ЭОС ФГБОУ ВО «МГТУ «СТАНКИН» [1].

Для заполнения базы данных требуемыми данными была написана специальная вспомогательная программа [26], производящая парсинг данных и приводящая их в нужную структуру и формат в SQLite (см. рис. 5.1).



	subject_name	lecture_count	sem_count	lab_count	lab_equipment
1	Безопасность жизнедеятельности	12	12	0	"
2	Компьютерная микроскопия	12	6	3	"
3	Компьютерные технологии в приборостроении	14	4	5	"
4	Экономическое обоснование инженерных решений	12	12	0	"
5	Организация и технология испытаний	10	4	3	"
6	Организация и планирование эксперимента	12	6	3	"
7	Системы качества	10	10	0	"
8	Интегрированные системы проектирования и управл...	12	6	3	"
9	Проектирование автоматизированных систем	14	6	4	"
10	Диагностика и надежность автоматизированных сист...	12	12	0	"
11	Организация и планирование автоматизированных п...	8	8	0	"
12	Вычислительные машины, системы и сети	12	4	4	"
13	Автоматные модели в системах управления	8	4	2	"
14	Проектирование роботов и робототехнических систем	12	0	4	"
15	Управление роботами и робототехническими систем...	10	4	3	"
16	Моделирование и исследование робототехнических ...	12	6	3	"
17	САПР робототехнических систем	10	0	3	"
18	Программное обеспечение мехатронных и робототе...	12	4	2	"
19	Компьютерное управление мехатронными системами	10	4	3	"
20	Проектирование мехатронных систем	12	0	4	"
21	Прикладная физическая культура	2	13	0	"
446	Деловые и межкультурные коммуникации	10	10	0	"

Рис. 5.1. Заполненная БД

5.2. СЕРВЕР

Как уже описывалось в предыдущей главе, в качестве основы вычислительной части проекта был выбран Polars. На основе его декларативного языка программирования был реализован алгоритм решения задачи. Все основные методы были обёрнуты в REST API благодаря FastAPI, а Swagger позволил упростить процесс написания будущего клиента.

Также были разработаны дополнительные инструменты визуализации расписания для упрощения дебага текущего состояния расписания, как например интерактивная визуализация всего расписания на семестр (см. рис. 5.2).

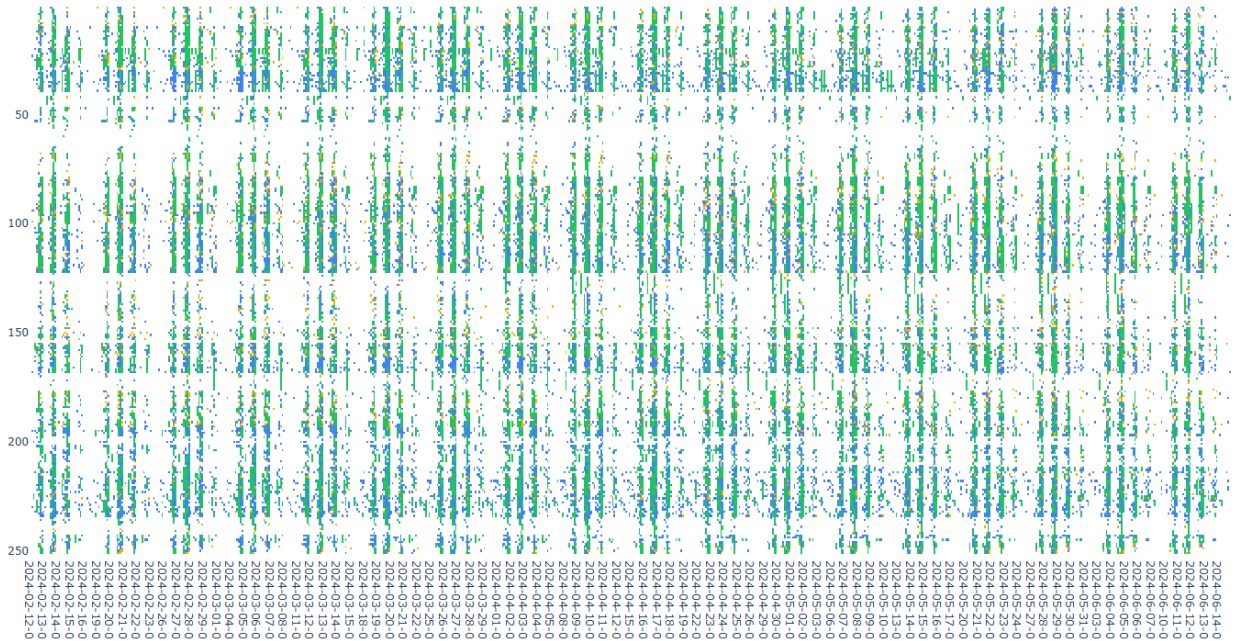


Рис. 5.2. Интерактивная визуализация расписания университета на семестр

5.2.1. Реализация WFC на языке Polars

Первым шагом является генерация первичного расписания, удовлетворяющего минимальным требованиям к расписанию учебных занятий. Этот шаг выполняется методом коллапса волновой функции.

Детальное описание алгоритма приведено ниже.

5.2.1.1. Подготовка таблиц

1. Загрузка из БД множеств групп G , преподавателей L , дисциплин S , а также соответствий «группа–дисциплина» (SubjectGroups) и «преподаватель–дисциплина» (SubjectLecturers).
2. Отфильтровка дисциплин без назначенного преподавателя. После этапа парсинга расписаний встречались предметы без преподавателей, и такие случаи требуется проработать в индивидуальном порядке.
3. Построение множества рабочих дней D : от начала до конца семестра согласно учебному плану, без воскресений.
4. Задание числа пар в день $T = 8$ и нумерация слотов $t \in \{0, \dots, T - 1\}$.

5. Формирование **суперпозиции** — декартова произведения:

$$\mathcal{C}_G = G \times D \times \{0, \dots, T-1\}, \quad \mathcal{C}_L = L \times D \times \{0, \dots, T-1\}.$$

Каждая ячейка (g, d, t) или (ℓ, d, t) изначально свободна (`subject_id` = $\emptyset$).

6. Вычисление **веса ячейки** $w(g, d, t)$:

$$w(g, d, t) = \log(1 + w_{\text{day}}(\text{weekday}(d))) \cdot w_{\text{slot}}(t),$$

с обнулением в праздники; для занятых ячеек $w(g, d, t) = 0$. Именно на этом шаге учитываются предпочтения преподавателей по нагрузке по дням недели, а для студентов двигается распределение — дневная или вечерняя форма обучения.

7. Вычисление **энтропии** группы $H(g)$ — суммарного числа оставшихся занятий по всем дисциплинам группы:

$$H(g) = \sum_{s \in S(g)} (n_{g,s}^{\text{лек}} + n_{g,s}^{\text{сем}} + n_{g,s}^{\text{лаб}}).$$

Аналогично определяется $H(\ell)$ для преподавателей.

5.2.1.2. Итерация WFC (один шаг `apply_one_step`)

Повторять, пока

$$\sum_{g \in G} \sum_{s \in S(g)} (n_{g,s}^{\text{лек}} + n_{g,s}^{\text{сем}} + n_{g,s}^{\text{лаб}}) > 0.$$

1. Выбор ячейки для коллапса. Среди свободных ячеек из $\mathcal{C}_G$ с $H(g) > 0$ выбирается множество с минимальной энтропией:

$$\mathcal{M} = \arg \min_{(g,d,t) \in \mathcal{C}_G, H(g) > 0} H(g).$$

Затем случайно выбирается $(g^*, d^*, t^*) \in \mathcal{M}$ с вероятностями, пропорциональными $\exp(w - \max w)$ (softmax по весу).

2. Сужение по преподавателям.

$$L^* = \{\ell \in L : \exists s \in S(g^*), (\ell, s) \in \text{SubjectLecturers}\},$$

после чего из этого множества исключаются преподаватели, уже занятые в слоте (d^*, t^*) :

$$L^* \leftarrow \{\ell \in L^* : \text{ячейка } (\ell, d^*, t^*) \text{ свободна}\}.$$

3. Коллапс дисциплины и типа занятия. Для каждой пары (s, c) , где $c \in \{\text{лек, сем, лаб}\}$, для которой $n_{g^*, s}^c > 0$ и существует $\ell \in L^*$ такой, что $(\ell, s) \in \text{SubjectLecturers}$, выбирается одна тройка (s^*, c^*, ℓ^*) взвешенным случайным выбором. Вес кандидата задаётся как

$$p(\ell) \propto \frac{1}{H(\ell) + \varepsilon}, \quad \varepsilon = 10^{-6}.$$

4. Определение целевых групп.

$$G_{\text{tar}} = \begin{cases} \{g^*\}, & c^* \in \{\text{сем, лаб}\}, \\ \{g \in G : \text{группа } g \text{ относится к тому же потоку, что и } g^*\}, & c^* = \text{лек}, \end{cases}$$

где «поток» — это группа групп с общим префиксом имени (например, ИДМ-24- для ИДМ-24-08), пересечённая с $S(g)$ для выбранной дисциплины s^* .

5. Назначение (коллапс). Для всех $g \in G_{\text{tar}}$ в ячейку (g, d^*, t^*) записывается:

$$(s^*, c^*, \ell^*), \quad \text{subject_type} \in \{0, 1, 2\} \Leftrightarrow \{\text{лек, сем, лаб}\}.$$

После этого заносится занятость в ячейку преподавателя (ℓ^*, d^*, t^*) .

6. Локальное распространение ограничений.

- Уменьшить $n_{g,s^*}^{c^*}$ на 1 для всех $g \in G_{\text{tar}}$;
- Уменьшить $H(g) \leftarrow \max(0, H(g) - 1)$ для всех $g \in G_{\text{tar}}$;
- Обнулить w для занятых ячеек.

7. Недельное распространение (flood fill). Пока

$$\min_{g \in G_{\text{tar}}} n_{g,s^*}^{c^*} > 0,$$

для $k = 1, 2, \dots, 52$ и $\sigma \in \{+1, -1\}$ выполняется попытка поставить то же занятие в $(g, d^* + \sigma k \text{ нед.}, t^*)$ для всех $g \in G_{\text{tar}}$, если:

- дата находится в пределах $[d_{\min}, d_{\max}]$;
- ячейки групп свободны (`subject_id` = $\emptyset$);
- преподаватель ℓ^* свободен в этом слоте.

При успешном назначении повторяется п. 6: декрементируются счётчики и обновляется энтропия.

8. Распределение аудиторий для занятий. Для всех аудиторий заранее вычисляется матрица кратчайших расстояний $\text{Dist}(a_i, a_j)$ (см. рис. 5.3).

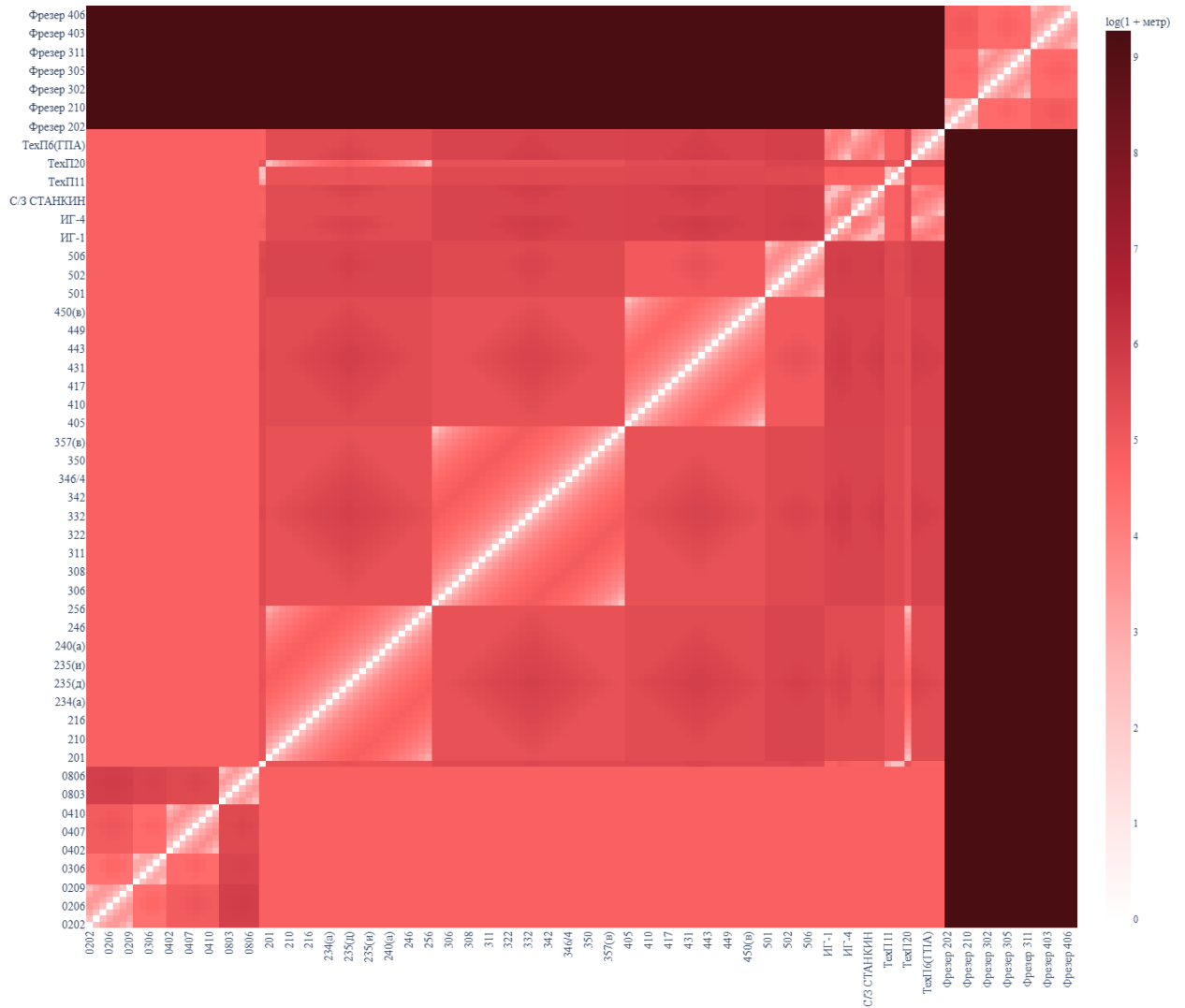


Рис. 5.3. Матрица примерных расстояний между аудиториями

9. Для назначенного занятия (g, d, t, s, c, ℓ) строится множество допустимых аудиторий:

$$A_{\text{adm}}(g, d, t, s, c) = \{a \in A : \text{cap}(a) \geq |G_{\text{tar}}|, \text{equip}(a) \supseteq \text{lab_equipment}(s, c), a$$

10. Выбор аудитории выполняется жадным образом с учётом уже назначенных занятий в тот же день d для группы g :

- если для дня d это первое занятие группы g , выбирается первая аудитория из A_{adm} , отсортированного в лексикографическом порядке по номеру аудитории;
- если в этот день уже назначена хотя бы одна аудитория a_{prev} для группы g , то множество A_{adm} сортируется по возрастанию

расстояния $\text{Dist}(a_{\text{prev}}, a)$, и выбирается первая допустимая аудитория.

11. Таким образом достигается двойная цель: минимизация числа используемых аудиторий за счёт устойчивого закрепления аудиторий за группами и минимизация перемещений внутри корпуса за счёт локальной оптимизации переходов между последовательными занятиями.

5.2.1.3. Завершение WFC

1. Остановка происходит, когда все счётчики $n^{\text{лек}}$, $n^{\text{сем}}$, $n^{\text{лаб}}$ обнуляются.
2. C_G и C_L сохраняются как итоговое расписание.

5.3. КЛИЕНТ

В роли клиента было написано веб-приложение на базе Nuxt 4 и Vue 3.5 (см. рис. 5.4), а технология progressive web application (PWA) позволяет установить сайт как нативное приложение на любое современное устройство, поддерживающее браузер. То есть генерацию расписания можно производить даже с некоторых холодильников или наручных часов.



Рис. 5.4. Клиент

5.4. ВЫВОД ПО ПЯТОЙ ГЛАВЕ

Была реализована программная основа сервиса для генерации расписания учебных занятий. Для системы была реализована трёхзвенная архитектура, обеспечивающая разделение ответственности между хранилищем данных, серверной логикой и пользовательским интерфейсом. Был подготовлен механизм заполнения БД на основе доступных источников данных.

На стороне сервера реализована бизнес-логика и REST API для взаимодействия с клиентской частью и внешними компонентами системы. На стороне клиента разработано веб-приложение, позволяющее удобно работать с сервисом через браузер. В результате получена работоспособная система, пригодная для дальнейшего развития и последующей интеграции в более полный процесс автоматизированного формирования расписания.

ГЛАВА 6. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

ГЛАВА 7. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

ЗАКЛЮЧЕНИЕ

1. Выпускная квалификационная работа посвящена решению задачи, заключающейся в ...
2. Анализ ...
3. На основании анализа... была разработана модель ...
4. Для реализации были выбраны ...
5. Реализация ...

СПИСОК ЛИТЕРАТУРЫ

1. Расписание занятий [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН». — URL: <https://edu.stankin.ru/course/view.php?id=11557> (дата обр. 18.12.2024).
2. Practices in timetabling in higher education institutions: a systematic review / R. A. Oude Vrielink [и др.] // PATAT 2016. — 2017.
3. Welsh, D. J. An upper bound for the chromatic number of a graph and its application to timetabling problems / D. J. Welsh, M. B. Powell // The Computer Journal. — 1967. — Т. 10, № 1. — С. 85—86.
4. A hybrid algorithm for the examination timetabling problem / L. T. Merlot [и др.] // PATAT 2002. — 2002.
5. A time-predefined local search approach to exam timetabling problems / E. Burke [и др.] // IIE Transactions. — 2004. — Т. 36, № 6. — С. 509—528.
6. Di Gaspero, L. Tabu search techniques for examination timetabling / L. Di Gaspero, A. Schaerf // PATAT 2000. — 2000.
7. Petrovic, S. A multiobjective optimisation technique for exam timetabling based on trajectories / S. Petrovic, Y. Bykov // PATAT 2002. — 2002.
8. Dowsland, K. A. Simulated annealing solutions for multi-objective scheduling and timetabling / K. A. Dowsland // Modern Heuristic Search Methods. — 1996. — С. 155—166.
9. A survey of search methodologies and automated system development for examination timetabling / R. Qu [и др.] // Journal of Scheduling. — 2009. — Т. 12, № 1. — С. 55—89.
10. Brailsford, S. C. Constraint satisfaction problems: Algorithms and applications / S. C. Brailsford, C. N. Potts, B. M. Smith // European Journal of Operational Research. — 1999. — Т. 119, № 3. — С. 557—581.

11. Corne, D. Evolutionary timetabling: Practice, prospects and work in progress / D. Corne, P. Ross, H.-L. Fang // UK planning and scheduling SIG workshop. — 1994.
12. Burke, E. K. A memetic algorithm for university exam timetabling / E. K. Burke, J. P. Newall, R. F. Weare // The international conference on the practice and theory of automated timetabling. — 1995.
13. Dorigo, M. Ant colony optimization theory: A survey / M. Dorigo, C. Blum // Theoretical Computer Science. — 2005. — Т. 344, № 2/3. — С. 243—278.
14. Al-Betar, M. A. A harmony search algorithm for university course timetabling / M. A. Al-Betar, A. T. Khader // Annals of Operations Research. — 2012. — Т. 194, № 1. — С. 3—31.
15. Российская Федерация. Трудовой кодекс Российской Федерации: ст. 112. Нерабочие праздничные дни / Российская Федерация. — 2001. — Нормативный правовой акт.
16. AloneCoder. Разбираемся с алгоритмом коллапса волновой функции [Электронный ресурс] / AloneCoder. — 2020. — URL: <https://habr.com/ru/companies/vk/articles/497590/> (дата обр. 01.03.2024) ; Текст: электронный.
17. Кливанский, Н. Н. Формирование расписания занятий высших учебных заведений / Н. Н. Кливанский // Образовательные ресурсы и технологии. — 2015. — Вып. 9, № 1.
18. SQLite Documentation [Электронный ресурс] / SQLite. — URL: <https://sqlite.org/docs.html> (дата обр. 14.04.2026).
19. Polars user guide [Электронный ресурс] / Polars. — URL: <https://docs.pola.rs/> (дата обр. 14.04.2026).
20. Automatic interactive API documentation [Электронный ресурс] / FastAPI. — URL: <https://fastapi.tiangolo.com/> (дата обр. 14.04.2026).

21. Introduction · Get Started with Nuxt v4 [Электронный ресурс] / Nuxt. — URL: <https://nuxt.com/docs/4.x/getting-started/introduction> (дата обр. 14.04.2026).

22. Информация по образовательным программам [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН». — URL: https://old.stankin.ru/pages/id_81/page_276 (дата обр. 18.12.2024).

23. Календарный учебный график [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН». — URL: https://old.stankin.ru/pages/id_81/page_257 (дата обр. 18.12.2024).

24. ebolblga/Grad-Work-Alpha [Электронный ресурс] / GitHub. — URL: <https://github.com/ebolblga/Grad-Work-Alpha> (дата обр. 18.12.2024).

25. Списки групп [Электронный ресурс] / ФГБОУ ВО «МГТУ «СТАНКИН». — URL: https://old.stankin.ru/pages/id_80/page_242 (дата обр. 18.12.2024).

26. ebolblga/Subject-Parser [Электронный ресурс] / GitHub. — URL: <https://github.com/ebolblga/Subject-Parser> (дата обр. 18.12.2024).

Приложение А.
Первое приложение

А.1. Раздел приложения

А.1.1. Подраздел приложения

Текст первого приложения...

А.2. Раздел приложения

А.2.2. Подраздел приложения

Текст первого приложения...

Б.1. Раздел приложения**Б.2. Раздел приложения****Б.3. Раздел приложения**

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Листинг Б.1.

Пример длинного листинга

```

^^I^^I#include <iostream>
^^I^^I#include <vector>
^^I^^I#include <cmath>
^^I^^Iusing namespace std;
^^I^^I
^^I^^Ivector<int> sieveOfEratosthenes(int n) {
^^I^^I^^Ivector<bool> isPrime(n + 1, true);
^^I^^I^^Ivector<int> primes;
^^I^^I^^I
^^I^^I^^IisPrime[0] = isPrime[1] = false;
^^I^^I^^I

```

```

^^I^^I^^Ifor (int i = 2; i * i ≤ n; i++) {
^^I^^I^^I^^Iif (isPrime[i]) {
^^I^^I^^I^^I^^Ifor (int j = i * i; j ≤ n; j += i) {
^^I^^I^^I^^I^^I^^IisPrime[j] = false;
^^I^^I^^I^^I^^I}
^^I^^I^^I^^I^^I}
^^I^^I^^I^^I}
^^I^^I^^I^^I
^^I^^I^^I^^Ifor (int i = 2; i ≤ n; i++) {
^^I^^I^^I^^I^^Iif (isPrime[i]) {
^^I^^I^^I^^I^^I^^Iprimes.push_back(i);
^^I^^I^^I^^I^^I}
^^I^^I^^I^^I}
^^I^^I^^I^^I
^^I^^I^^I^^Ireturn primes;
^^I^^I^^I}
^^I^^I^^I
^^I^^I^^Iint main() {
^^I^^I^^I^^Iint limit;
^^I^^I^^I^^Icout << "Введите верхнюю границу для поиска простых чисел: ";
^^I^^I^^I^^Icin >> limit;
^^I^^I^^I^^I
^^I^^I^^I^^Ivector<int> primes = sieveOfEratosthenes(limit);
^^I^^I^^I^^I
^^I^^I^^I^^Icout << "Найдено простых чисел: " << primes.size() << endl;
^^I^^I^^I^^Icout << "Все простые числа до " << limit << ":" << endl;
^^I^^I^^I^^I
^^I^^I^^I^^Ifor (int i = 0; i < primes.size(); i++) {
^^I^^I^^I^^I^^Icout << primes[i] << " ";
^^I^^I^^I^^I^^Iif ((i + 1) % 10 == 0) cout << endl; // 10 чисел в строке
^^I^^I^^I^^I}
^^I^^I^^I^^Icout << endl;
^^I^^I^^I^^I
^^I^^I^^I^^Ireturn 0;
^^I^^I^^I}^^I

```
